

NoSQL Databases Types & CAP Theorem

Chapter 06 - Sub-Chapter 01

NoSQL Databases Series

What is NoSQL?	Why it was invented and what problems it solves
Four NoSQL Types	Key-Value, Document, Wide-Column, Graph
CAP Theorem	Consistency vs Availability in distributed systems
SQL vs NoSQL	Choosing the right tool with real examples
Consistency Models	Strong, Eventual, Read-your-own-writes

1 What is NoSQL?

And why was it invented?

Imagine you run a small shop and keep all customer records in a neat Excel spreadsheet with fixed columns: Name, Phone, Address, Email. This works great -- until one customer has three phone numbers, another has no email, and a third has a social media handle instead. Your fixed columns break down.

This is exactly the problem that drove the creation of NoSQL. As the internet grew in the 2000s, companies like Google, Amazon, and Facebook faced data that was:

```
// Traditional SQL problem at internet scale:

// Facebook user profile -- rigid SQL table
CREATE TABLE users (
  id          BIGINT PRIMARY KEY,
  name        VARCHAR(100),
  email       VARCHAR(100),
  phone       VARCHAR(20)    -- what if user has 3 phones?
  -- you cannot add a column per user -- table has 3 BILLION rows
);

// Problems:
// 1. Schema change on 3B rows = hours of downtime
// 2. User in India has "aadhar_number", US user has "ssn" -- different fields
// 3. One server cannot hold 3 billion rows -- need to split across 1000 servers
// 4. JOINing across 1000 servers is impossibly slow

// NoSQL solution -- each user document stores exactly what they have:
{
  "id": "u_001",
  "name": "Manish Sharma",
  "phones": ["+91-9876543210", "+91-9123456789"], // array -- flexible
  "aadhar": "1234-5678-9012",                  // India-specific field
  "preferences": { "theme": "dark", "language": "hi" }
  // no "ssn" field -- because this user doesn't have one
}
```

NoSQL does NOT mean 'No SQL ever'. It means '**Not Only SQL**' -- a family of databases designed for specific use cases where relational tables are a poor fit.

Key Rules

- NoSQL was invented to solve problems SQL databases struggle with at internet scale.
- Key problems: rigid schema, single-server limits, slow JOINS across distributed nodes.
- NoSQL = Not Only SQL -- it complements SQL rather than replacing it.
- Google invented Bigtable (2006), Amazon invented Dynamo (2007) -- these inspired modern NoSQL.
- Today most large apps use BOTH SQL (for transactions) and NoSQL (for scale/flexibility).

Q1. Why was NoSQL invented?

SQL databases struggle with flexible schemas, massive horizontal scale, and storing hierarchical data. NoSQL trades some SQL features (like JOINS and ACID) for speed, scale, and flexibility.

Q2. Does NoSQL replace SQL?

No. NoSQL complements SQL. Most real applications use both -- SQL for financial transactions and NoSQL for user sessions, product catalogs, real-time feeds, etc.

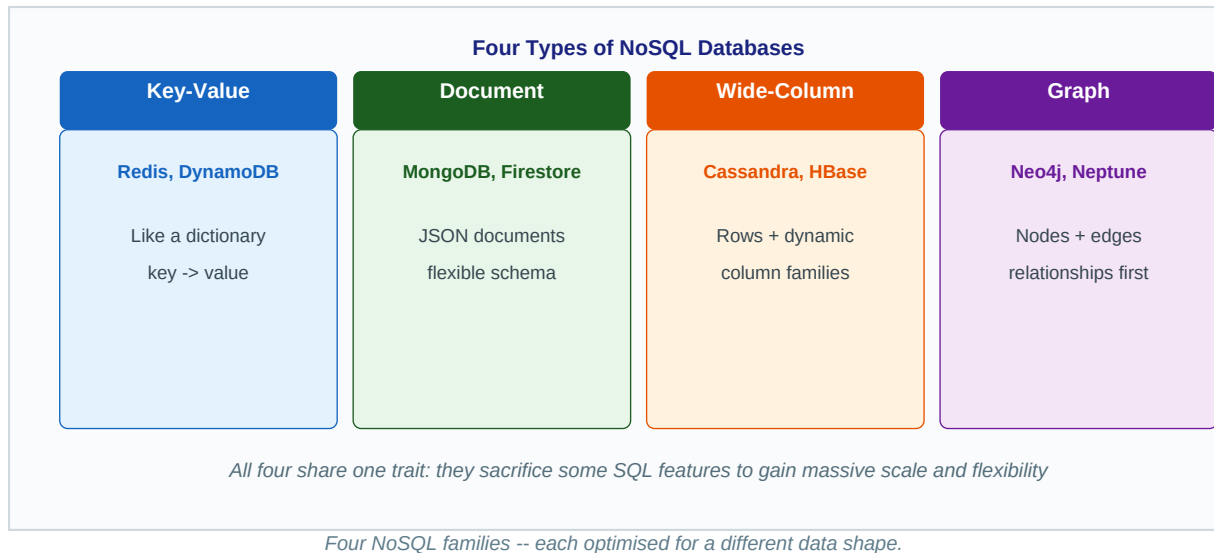
Q3. Who invented NoSQL databases?

Google published the Bigtable paper in 2006, Amazon published the Dynamo paper in 2007. These inspired open-source databases like HBase, Cassandra, MongoDB, and DynamoDB.

2 Four Types of NoSQL

Key-Value, Document, Wide-Column, Graph

Think of NoSQL as four different filing systems, each built for a different job. Using the wrong one is like using a photo album to store a phone book.



Type 1 -- Key-Value Store (fastest, simplest):

Analogy: A coat check at a restaurant. You hand in your coat, get a ticket (key). Show the ticket later, get your coat (value). The attendant does not care what is in your coat -- they just store and retrieve it by ticket number.

```
// Redis -- Key-Value example
// Real use case: Store user session after login

// User logs in -- store session for 30 minutes
SET session:user_42 '{"userId":42,"name":"Manish","role":"admin"}' EX 1800
//   ^key                ^value (JSON string)                ^expire in 1800 sec

// Any server in the cluster fetches the session instantly
GET session:user_42
// -> {"userId":42,"name":"Manish","role":"admin"}

// User logs out -- delete session
DEL session:user_42

// Why not SQL for this?
// SELECT * FROM sessions WHERE id = 'user_42'
// -> Goes to disk, hits indexes, takes 5-50ms
// Redis GET -> in-memory, takes 0.1ms -- 50-500x faster
```

Type 2 -- Document Store (flexible JSON):

Analogy: A filing cabinet where each folder can contain any papers in any format. Folder 1 has 3 pages, Folder 2 has 20 pages with photos -- no fixed structure required.

```
// MongoDB -- Document Store example
// Real use case: Product catalog for an e-commerce site

// Electronics product -- has "specs" with technical details
{
```

```

    "_id": "prod_001",
    "name": "Samsung Galaxy S24",
    "category": "smartphone",
    "price": 79999,
    "specs": {
      "ram": "8GB", "storage": "256GB",
      "camera": "50MP", "battery": "4000mAh"
    },
    "tags": ["android", "5g", "samsung"]
  }

// Book product -- completely different fields, same collection
{
  "_id": "prod_002",
  "name": "Clean Code",
  "category": "book",
  "price": 599,
  "author": "Robert C. Martin",
  "isbn": "978-0132350884",
  "pages": 431
  // no "specs" field -- books don't have RAM
}

// SQL would force both rows into the same rigid table
// MongoDB stores each document independently -- schema per document

```

Type 3 -- Wide-Column Store (for time-series, IoT, analytics):

Analogy: A spreadsheet where each row can have completely different columns. Row 1 has columns for Jan-Mar data. Row 2 has columns for sensor readings every second. Rows can have millions of columns.

```

// Cassandra -- Wide-Column example
// Real use case: IoT sensor readings for a fleet of vehicles

CREATE TABLE vehicle_telemetry (
  vehicle_id TEXT,
  recorded_at TIMESTAMP,
  speed DOUBLE,
  fuel_level DOUBLE,
  latitude DOUBLE,
  longitude DOUBLE,
  PRIMARY KEY (vehicle_id, recorded_at) -- partition by vehicle, sort by time
) WITH CLUSTERING ORDER BY (recorded_at DESC);

// Insert a reading
INSERT INTO vehicle_telemetry
  (vehicle_id, recorded_at, speed, fuel_level, latitude, longitude)
VALUES ('TRK-001', '2024-01-15 10:30:00', 65.5, 78.2, 28.6139, 77.2090);

// Get last 100 readings for a vehicle -- blazing fast (no JOIN needed)
SELECT * FROM vehicle_telemetry
WHERE vehicle_id = 'TRK-001'
LIMIT 100;
// This query works even across 1000 Cassandra nodes -- always fast

```

Type 4 -- Graph Database (relationships are first-class):

Analogy: A map of a city. Cities are nodes, roads are edges. Finding the shortest route is a graph problem -- SQL would need dozens of JOINS for the same query.

```

// Neo4j -- Graph Database example
// Real use case: Social network -- find friends of friends

```

```
// Create nodes (people)
CREATE (manish:Person {name: 'Manish', city: 'Delhi'})
CREATE (priya:Person {name: 'Priya', city: 'Mumbai'})
CREATE (raj:Person {name: 'Raj', city: 'Delhi'})

// Create relationships (edges)
CREATE (manish)-[:FRIENDS_WITH]->(priya)
CREATE (priya)-[:FRIENDS_WITH]->(raj)

// Find friends of Manish's friends (2 hops) -- 1 simple query
MATCH (manish:Person {name:'Manish'})-[:FRIENDS_WITH*2]->(fof)
RETURN fof.name;
// -> Raj (Manish -> Priya -> Raj)

// In SQL this requires multiple JOINS:
// SELECT u3.name FROM users u1
// JOIN friendships f1 ON u1.id = f1.user_id
// JOIN friendships f2 ON f1.friend_id = f2.user_id
// JOIN users u3 ON f2.friend_id = u3.id
// WHERE u1.name = 'Manish'
// -- gets worse with each hop (3 hops = 4 JOINS, very slow at scale)
```

Key Rules

- Key-Value: Fastest. Use for caching, sessions, rate limiting. Redis is the king.
- Document: Most flexible. Use for product catalogs, user profiles, content. MongoDB leads.
- Wide-Column: Best for time-series and IoT. Billions of writes/day. Cassandra is standard.
- Graph: Best for relationships. Social networks, fraud detection, recommendations. Neo4j leads.
- Wrong choice = bad performance. Always choose the DB that matches your data shape.

Q1. When should I use a Key-Value store?

Use it for caching, session storage, rate limiting counters, and leaderboards. If your access pattern is always 'give me X by its ID', Key-Value is perfect.

Q2. When should I use a Document store?

When each record has a different structure (product catalog, user profiles, blog posts). MongoDB lets each document have different fields.

Q3. When should I use a Wide-Column store?

When you write millions of events per second and query by a partition key (device ID, user ID) and time range. Cassandra and HBase excel here.

Q4. When should I use a Graph database?

When relationships between data are as important as the data itself -- social networks, fraud detection, recommendation engines, knowledge graphs.

3 CAP Theorem

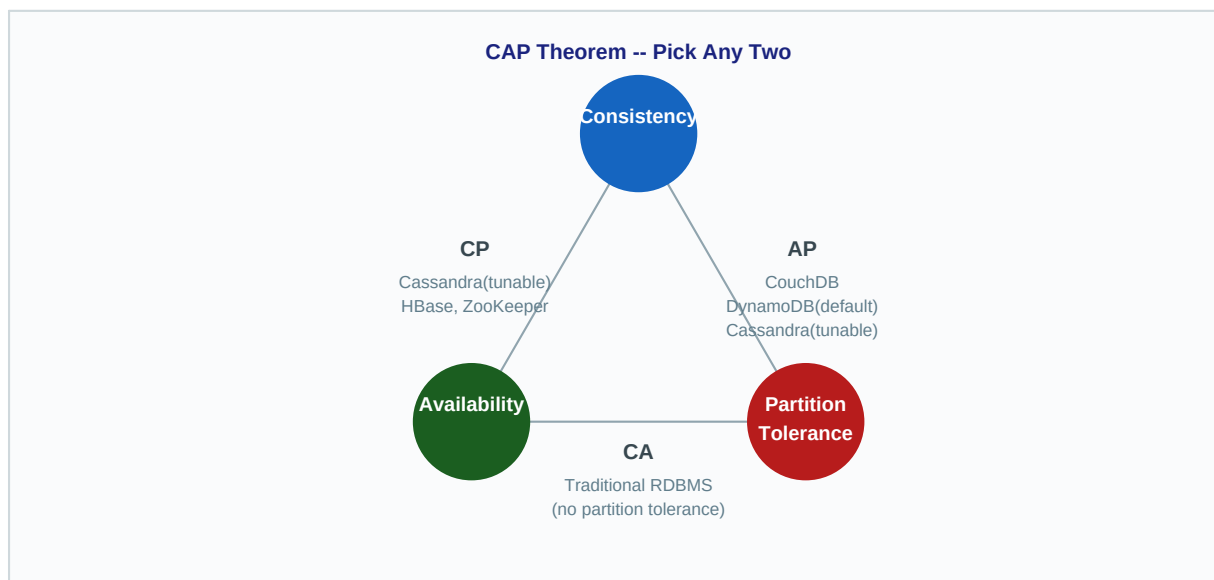
The fundamental trade-off every distributed database makes

Imagine you run a bank with two branches -- Delhi and Mumbai. Both branches share customer data. Now the internet connection between Delhi and Mumbai breaks (this is called a **network partition**). What do you do?

Option A: Stop all transactions at both branches until the connection is restored (choose Consistency -- every customer sees the same data -- but sacrifice Availability).

Option B: Let both branches keep serving customers independently, then reconcile the differences later (choose Availability -- always serve -- but risk Consistency -- Delhi and Mumbai may show different balances).

This dilemma is exactly the **CAP Theorem**, proven by Eric Brewer in 2000: a distributed database can guarantee at most **two** of three properties simultaneously.



CAP Theorem -- every distributed database picks two. Network partitions always happen in practice, so the real choice is C vs A.

The three properties explained:

Property	Plain English	Real world analogy
Consistency (C)	Every read returns the most recent write -- all nodes see same data	ATM always shows your exact balance -- never stale
Availability (A)	Every request gets a response -- system never refuses	Shop always serves customers, even if stock info is slightly old
Partition Tolerance (P)	System keeps working even if network between nodes fails	Delhi and Mumbai branches work independently if phone line cuts

Practical database choices:

```
// CP Systems -- Consistency + Partition Tolerance
// Sacrifice: Availability (may reject requests during partition)
// Use when: Data correctness is critical -- banking, inventory, bookings

// HBase example -- if partition detected, rejects writes to stay consistent
```

```
// ZooKeeper -- used for distributed coordination, must never have split-brain
// MongoDB (default write concern) -- waits for majority acknowledgment

// AP Systems -- Availability + Partition Tolerance
// Sacrifice: Consistency (may return stale data during partition)
// Use when: Always responding matters more than perfect accuracy

// DynamoDB (eventual consistency mode)
// Cassandra (default) -- always accepts writes, syncs later
// CouchDB -- each node works independently, merges later

// Example -- Cassandra AP behaviour during partition:
// Node A (Delhi) writes: user balance = 5000
// Network splits -- Node B (Mumbai) does not get the update
// User queries Node B -- gets old balance = 4500 (stale read)
// Partition heals -- Cassandra reconciles: balance = 5000 (latest wins)
// This is called "eventual consistency" -- data converges, but not instantly

// CA Systems -- Consistency + Availability
// Only possible on a single node (no partition tolerance)
// Traditional RDBMS: MySQL, PostgreSQL (single server)
// NOT suitable for distributed systems
```

PACELC -- the modern extension of CAP:

CAP only describes behaviour during a network partition. But what about **normal operation**? The PACELC model adds: even when no partition exists (E), there is a trade-off between **Latency (L)** and **Consistency (C)**.

```
// PACELC: If Partition -> choose A or C
//           Else (normal) -> choose L or C

// DynamoDB: PA/EL
//   During partition: chooses Availability
//   Normal operation: chooses low Latency (may return slightly stale data)

// HBase: PC/EC
//   During partition: chooses Consistency
//   Normal operation: chooses Consistency (waits for all replicas)

// Cassandra: PA/EL by default, but tunable per query:
ConsistencyLevel level = ConsistencyLevel.QUORUM; // tunable
// QUORUM = majority of nodes must agree -- trades latency for consistency
// ONE    = fastest, least consistent
// ALL    = slowest, fully consistent
```

Key Rules

- CAP Theorem: a distributed DB can only guarantee 2 of: Consistency, Availability, Partition Tolerance.
- Partition Tolerance is mandatory in real distributed systems -- network failures always happen.
- So the real choice is: CP (correct but may be unavailable) vs AP (always available but may be stale).
- Cassandra is AP by default but tunable -- you can dial up consistency per query.
- PACELC extends CAP: even without partitions, you choose between latency and consistency.
- Banking needs CP. Social feeds need AP. Know your requirement before choosing a DB.

Q1. What does CAP stand for?

Consistency (every read gets the latest write), Availability (every request gets a response), Partition Tolerance (works even if network splits). A distributed DB can only guarantee two.

Q2. Why can't we have all three?

If a network partition happens and you want Consistency, you must stop serving requests (lose Availability). If you keep serving requests (Availability), you may return stale data (lose Consistency).

Q3. What is eventual consistency?

AP databases accept writes on any available node and sync them to other nodes later. Data will be consistent eventually -- but there is a window where different nodes show different values.

Q4. Is Cassandra CP or AP?

AP by default. But it is tunable -- you can set consistency level to QUORUM or ALL per query to get stronger consistency at the cost of latency.

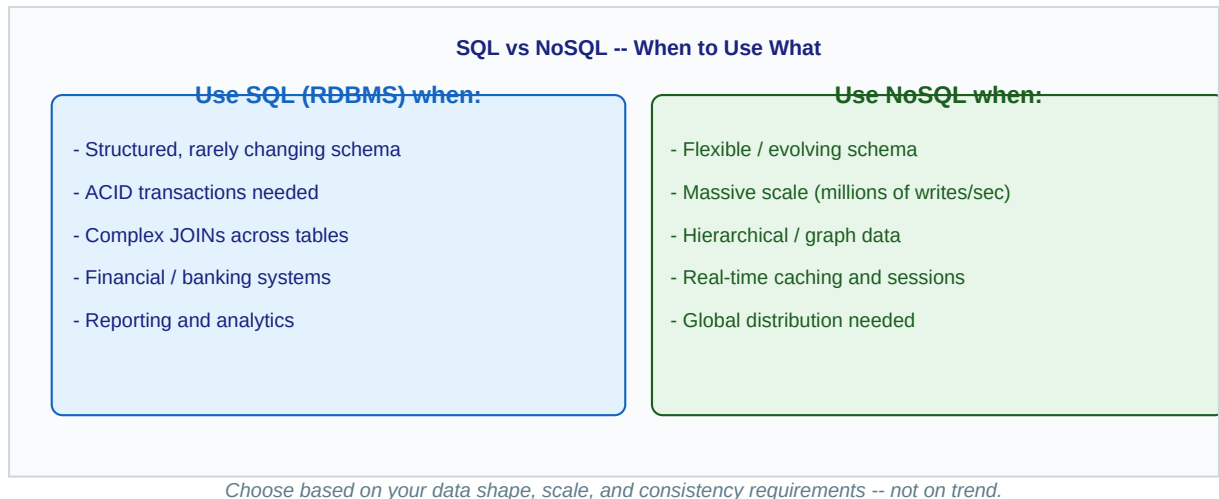
Q5. When does a network partition happen?

Any time two nodes cannot communicate -- network cable cut, switch failure, data centre outage, cloud availability zone going down. In large clusters this happens regularly.

4 SQL vs NoSQL

Choosing the right tool for the job

A common mistake is to treat NoSQL as 'better than SQL' or 'the modern way'. Both are the right answer -- for different problems. A hammer is not better than a screwdriver -- it depends on the job.



Realistic comparison with a real application:

Scenario: You are building a food delivery app (like Swiggy/Zomato). Let us see which database fits each part:

```
// Food delivery app -- multiple databases, each doing what it does best

// 1. Orders, Payments -- SQL (PostgreSQL)
//   Because: ACID transactions, cannot lose money, need JOINS
CREATE TABLE orders (
  id          BIGSERIAL PRIMARY KEY,
  user_id     BIGINT REFERENCES users(id),
  restaurant_id BIGINT REFERENCES restaurants(id),
  total       DECIMAL(10,2),
  status      VARCHAR(20),
  created_at  TIMESTAMP DEFAULT NOW()
);
// If payment fails, order row is rolled back -- ACID saves you

// 2. User sessions / auth tokens -- Redis (Key-Value)
//   Because: Needs to be fast (every API call checks session)
SET session:tok_abc123 '{"userId":42,"role":"customer"}' EX 3600
GET session:tok_abc123 // 0.1ms -- checked on every request

// 3. Restaurant menus -- MongoDB (Document)
//   Because: Every restaurant has different fields
{
  "restaurant_id": "rest_001",
  "name": "Punjabi Dhaba",
  "menu": [
    {"item": "Dal Makhani", "price": 280, "veg": true, "spice_level": "medium"},
    {"item": "Butter Chicken", "price": 380, "veg": false, "contains": ["dairy", "gluten"]}
  ],
  "timings": {"open": "11:00", "close": "23:00"},
  "features": ["pure_veg", "home_delivery", "dine_in"]
}
```

```
// 4. Delivery tracking / GPS events -- Cassandra (Wide-Column)
//   Because: 100,000 drivers sending location every 5 seconds = 20M writes/min
INSERT INTO driver_location (driver_id, recorded_at, lat, lng, speed)
VALUES ('drv_001', NOW(), 28.6139, 77.2090, 42.5);

// 5. Recommendations -- Neo4j (Graph)
//   Because: "Users who ordered X also ordered Y" is a graph traversal
MATCH (u:User)-[:ORDERED]->(i:Item)-[:ORDERED]-(other:User)-[:ORDERED]->(rec:Item)
WHERE u.id = 42 AND NOT (u)-[:ORDERED]->(rec)
RETURN rec.name, count(*) AS score ORDER BY score DESC LIMIT 5
```

Schema flexibility -- the biggest practical difference:

```
// SQL: Adding a field to an existing table with millions of rows

// Bad approach on large tables:
ALTER TABLE users ADD COLUMN profile_photo_url VARCHAR(500);
// On a table with 50M rows this can lock the table for HOURS
// Workaround: use online schema change tools (pt-online-schema-change)

// MongoDB: Adding a field -- just start writing it
// No migration needed -- new documents have the field, old ones don't
db.users.updateOne(
  { _id: ObjectId("user_001") },
  { $set: { profile_photo_url: "https://cdn.example.com/photos/u1.jpg" } }
);
// Old users without the field -- application handles null gracefully
// Zero downtime, zero table lock, zero migration script
```

Criteria	Choose SQL	Choose NoSQL
Data structure	Fixed, well-defined schema	Flexible, varying fields
Scale	Vertical (bigger server)	Horizontal (more servers)
Transactions	Multi-table ACID required	Single-document or eventual OK
Queries	Complex JOINS, aggregations	Simple lookups by key or range
Consistency	Strong consistency always	Eventual consistency acceptable
Team	SQL expertise available	Willing to learn NoSQL patterns
Examples	Banking, ERP, HR systems	Social, IoT, gaming, catalogs

Key Rules

- Use SQL when data is structured, relations matter, and ACID is required.
- Use NoSQL when schema is flexible, scale is massive, or data is hierarchical.
- Most real-world apps use both -- polyglot persistence is the norm, not the exception.
- The biggest NoSQL mistake: using it for everything, including things SQL does better.
- Start with SQL for new projects -- add NoSQL only when you hit a specific limitation.

Q1. Can I use NoSQL for financial transactions?

Generally no. Financial transactions need ACID guarantees across multiple records. Use SQL (PostgreSQL, MySQL) for financial data and NoSQL for supplementary data like notifications or audit logs.

Q2. Does Google use SQL or NoSQL?

Both. Google uses Spanner (globally distributed SQL), Bigtable (wide-column NoSQL), Firestore (document NoSQL), and more. Large companies always use multiple databases.

Q3. Is MongoDB better than MySQL?

Neither is universally better. MongoDB excels for flexible document data. MySQL excels for relational, transactional data. Choose based on your specific use case.

Q4. What is polyglot persistence?

Using multiple database types in one application -- SQL for orders, Redis for sessions, MongoDB for catalogs, Cassandra for events. Each database does what it does best.

5 Consistency Models

Strong, Eventual, and everything in between

When you write data to a distributed database, it gets copied to multiple servers (replicas) for durability. The **consistency model** defines: how quickly does every replica show the new value, and what does a reader see in the meantime?

Strong Consistency -- everyone sees the latest write immediately:

```
// Strong consistency example -- bank balance
// Node A and Node B are replicas

// Step 1: Alice writes balance = 5000 to Node A
// Step 2: Node A synchronously replicates to Node B BEFORE confirming write
// Step 3: Write confirmed to Alice
// Step 4: Bob reads from Node B -> gets 5000 (always latest)

// Code example using HBase (strongly consistent by default):
import org.apache.hadoop.hbase.client.*;

Put put = new Put(Bytes.toBytes("alice"));
put.addColumn(
    Bytes.toBytes("account"), // column family
    Bytes.toBytes("balance"), // column
    Bytes.toBytes("5000")     // value
);
table.put(put); // blocks until ALL replicas confirm

Get get = new Get(Bytes.toBytes("alice"));
Result result = table.get(get);
// Always returns 5000 -- even if querying a different replica
```

Eventual Consistency -- replicas sync after a short delay:

```
// Eventual consistency example -- social media like count
// Millions of likes per second -- strong consistency would be too slow

// DynamoDB eventual consistency read (default -- faster, cheaper):
import software.amazon.awssdk.services.dynamodb.DynamoDbClient;
import software.amazon.awssdk.services.dynamodb.model.*;

GetItemRequest request = GetItemRequest.builder()
    .tableName("Posts")
    .key(Map.of("postId", AttributeValue.builder().s("post_001").build()))
    .consistentRead(false) // eventual consistency -- may return stale data
    .build();

GetItemResponse response = dynamoDb.getItem(request);
// Might return likeCount=9999 even if actual is 10000
// (replica hasn't received the latest update yet)
// But within milliseconds to seconds, all replicas will sync

// DynamoDB strongly consistent read (slower, costs 2x read units):
GetItemRequest strongRequest = GetItemRequest.builder()
    .tableName("Posts")
    .key(Map.of("postId", AttributeValue.builder().s("post_001").build()))
    .consistentRead(true) // always returns latest -- costs more
    .build();

// For social media likes -- eventual is fine
// For bank balance -- always use strong consistency
```

Read-your-own-writes consistency:

A common middle ground: after YOU write something, YOU always see your own write. Others may see the old value briefly. Used in most social media and e-commerce.

```
// Cassandra -- tunable consistency level per operation
import com.datastax.oss.driver.api.core.CqlSession;
import com.datastax.oss.driver.api.core.DefaultConsistencyLevel;

// Write with QUORUM -- majority of replicas must confirm
// (ensures read-your-writes when reading at QUORUM too)
SimpleStatement write = SimpleStatement.builder(
    "UPDATE user_profiles SET bio = ? WHERE user_id = ?")
    .addPositionalValues("Software Engineer at Acme", "user_42")
    .setConsistencyLevel(DefaultConsistencyLevel.QUORUM) // majority must agree
    .build();
session.execute(write);

// Read with QUORUM -- guaranteed to see the write above
SimpleStatement read = SimpleStatement.builder(
    "SELECT bio FROM user_profiles WHERE user_id = ?")
    .addPositionalValues("user_42")
    .setConsistencyLevel(DefaultConsistencyLevel.QUORUM)
    .build();
Row row = session.execute(read).one();
// row.getString("bio") -> "Software Engineer at Acme" -- guaranteed
```

Key Rules

- Strong consistency: every read sees the latest write. Slower but safe. Use for financial data.
- Eventual consistency: replicas sync after a delay. Faster, more available. Use for social data.
- Read-your-own-writes: you always see your own writes. Others may see old data briefly.
- Cassandra lets you choose consistency level per query -- most flexible NoSQL model.
- For most apps: use eventual consistency by default, strong only where correctness is critical.

Q1. What is eventual consistency in simple terms?

After a write, not all nodes update immediately. Within a short time (milliseconds to seconds) all nodes converge to the same value. Like a WhatsApp message that shows 'delivered' to all recipients eventually.

Q2. When is eventual consistency dangerous?

For bank balances, inventory counts (only 1 item left), medical records, or any data where stale reads cause real harm. Use strong consistency for these.

Q3. What is the consistency level QUORUM in Cassandra?

QUORUM means a majority of replicas (e.g., 2 out of 3) must acknowledge a write before it is confirmed, and a read must check a majority before returning. This balances availability and consistency.